

Install Required Libraries

```
pip install tensorflow numpy
```

Import Libraries

```
import numpy as np
import tensorflow as tf

from tensorflow.keras.models import Sequential, load_model, model_from_json
from tensorflow.keras.layers import Dense
from tensorflow.keras.callbacks import ModelCheckpoint
```

Task 1: Train a Sentiment Classifier and Save as .h5

Create a Small Dataset

```
import numpy as np

# Features:
# Review Length, Positive Word Score

X = np.array([
    [9,8],
    [8,7],
    [7,6],
    [2,1],
    [3,2],
    [1,1],
    [8,9],
    [9,9],
    [2,3],
    [1,2]
], dtype=float)

# Labels
# 1 = Positive
# 0 = Negative

y = np.array([
    1,1,1,
```

```
0,0,0,  
1,1,  
0,0  
)
```

Build Model

```
model = Sequential([  
  
    Dense(16, activation="relu", input_shape=(2,)),  
  
    Dense(8, activation="relu"),  
  
    Dense(1, activation="sigmoid")  
  
)
```

```
model.compile(  
  
    optimizer="adam",  
  
    loss="binary_crossentropy",  
  
    metrics=["accuracy"]  
  
)
```

Train Model

```
model.fit(  
  
    X,  
  
    y,  
  
    epochs=100,  
  
    validation_split=0.2,  
  
    verbose=1  
  
)
```

Save Model

```
model.save("sentiment_model.h5")
```

```
print("Model Saved Successfully!")
```

Task 2: Load Model and Predict New Reviews

Load Saved Model

```
loaded_model = load_model("sentiment_model.h5")
```

New Reviews

```
new_reviews = np.array([
```

```
    [9,9], # Very Positive
```

```
    [2,2], # Negative
```

```
    [7,8]  # Positive
```

```
], dtype=float)
```

Predict

```
predictions = loaded_model.predict(new_reviews)
```

```
for i, pred in enumerate(predictions):
```

```
    sentiment = "Positive" if pred > 0.5 else "Negative"
```

```
    print(f"Review {i+1}: {sentiment} ({pred[0]:.3f})")
```

Example Output

```
Review 1: Positive (0.985)
```

```
Review 2: Negative (0.041)
```

```
Review 3: Positive (0.923)
```

Task 3: Model Checkpointing

```
checkpoint = ModelCheckpoint(
```

```
    "best_model.keras",
```

```
    monitor="val_accuracy",
```

```
    save_best_only=True,
```

```
    mode="max",
```

```
    verbose=1
```

```
)
```

```
history = model.fit(
```

```
    X,
```

```
    y,
```

```
    validation_split=0.2,
```

```
    epochs=100,
```

```
    callbacks=[checkpoint]
```

```
)
```

Explanation

Every time **validation accuracy improves**, Keras saves the best model automatically.

Task 4: Export Model for Deployment

Save as TensorFlow SavedModel

```
model.export("saved_sentiment_model")
```

If you are using an older TensorFlow/Keras version that does not support `model.export()`, use:

```
model.save("saved_sentiment_model")
```

This creates a TensorFlow SavedModel directory.

Folder Structure

saved_sentiment_model/

assets/

variables/

saved_model.pb

Why SavedModel?

TensorFlow SavedModel stores the complete model (architecture, weights, and serving signatures), making it easy to deploy to web, cloud, or mobile applications.

Task 5: Save Architecture and Weights Separately

Save Architecture

```
model_json = model.to_json()
```

```
with open("model_architecture.json", "w") as file:  
    file.write(model_json)
```

Save Weights

```
model.save_weights("model.weights.h5")
```

Load Architecture

```
with open("model_architecture.json", "r") as file:  
    loaded_json = file.read()
```

```
loaded_model = model_from_json(loaded_json)
```

Load Weights

```
loaded_model.load_weights("model.weights.h5")
```

Compile Loaded Model

```
loaded_model.compile(  
  
    optimizer="adam",  
  
    loss="binary_crossentropy",  
  
    metrics=["accuracy"]  
  
)
```

Compare Predictions

```
original_predictions = model.predict(new_reviews)  
  
loaded_predictions = loaded_model.predict(new_reviews)  
  
print("Original Predictions")  
  
print(original_predictions)  
  
print()  
  
print("Loaded Model Predictions")  
  
print(loaded_predictions)
```

Verify

```
print(  
  
    np.allclose(  
  
        original_predictions,  
  
        loaded_predictions  
  
    )  
  
)
```

Output

True

This confirms that the loaded model produces the same predictions as the original model (allowing for tiny floating-point differences).

Project Folder Structure

Sentiment_Project/

|— sentiment_model.h5

|— best_model.keras

|— model_architecture.json

|— model.weights.h5

|— saved_sentiment_model/

| |— assets/

| |— variables/

| └— saved_model.pb

|— sentiment_classifier.py